

Applications of Artificial Intelligence

Unit 7 Consolidate - Individual Activities

Activity 1: Perceptron Calculation

Perform the following calculations manually to understand how a perceptron works:

1. Given inputs [1.5, 2.0, 0.5] and weights [0.8, -0.2, 1.0] with bias 0.5:

Response:

Weighted sum: $(1.5 \times 0.8) + (2.0 \times -0.2) + (0.5 + 1.0) + 0.5 = 1.8$

Sigmoid activation function = $f(1.8) = 1 / (1 + e^{-1.8}) = 0.8581$

2. Change one weight to double its value. Recalculate and observe the difference in output.

Response:

Doubling the value of the first weight (from 0.8 to 1.6) changes the calculation as follows:

Weighted sum: $(1.5 \times 1.6) + (2.0 \times -0.2) + (0.5 + 1.0) + 0.5 = 3.0$

Sigmoid activation function = $f(3.0) = 1 / (1 + e^{-3.0}) = 0.9526$

The output increases by 0.0945 (calculated as $0.9526 - 0.8581$). A heavier weight for a positive input will result in a larger weighted sum, and a larger sigmoid activation function.

3. Reflection: What does this tell you about the sensitivity of neural networks to weight changes?

Higher weights result in higher sensitivity to changes. As inputs are multiplied by weights, small weight adjustments can result in larger changes in output.

Very large weights (like 3.0 above) can push the sigmoid activation function towards its upper limit of 1.

Activity 2: Function Approximation

Draw a sigmoid function $f(x) = 1 / (1 + e^{-x})$

Draw a ReLU function $f(x) = \max(0, x)$

Draw a tanh function $f(x) = (e^x - e^{-x}) / (e^x + e^{-x})$

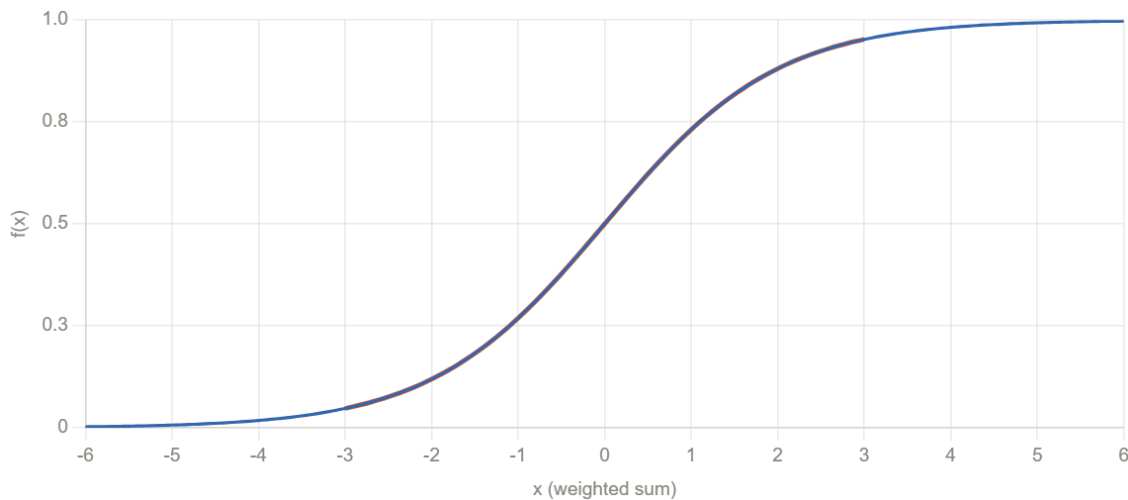
For each function, identify:

Input range that produces significant output changes

Where the function saturates (if applicable)

Potential advantages/disadvantages in neural networks

Sigmoid function for $f(x) = 1 / (1 + e^{-x})$



a.) Input range that produces significant output changes

The purple-colored part of the line ($-3 < x < 3$) represents the input range for the weighted sum that produces the most significant output changes, as it is the steepest part of the curve.

b.) Where the function saturates (if applicable)

The function saturates at $x > 3$ ($f(x) = 1$), and $x < -3$ ($f(x) = 0$). These are the flatter parts of the curve.

c.) Potential advantages/disadvantages in neural networks

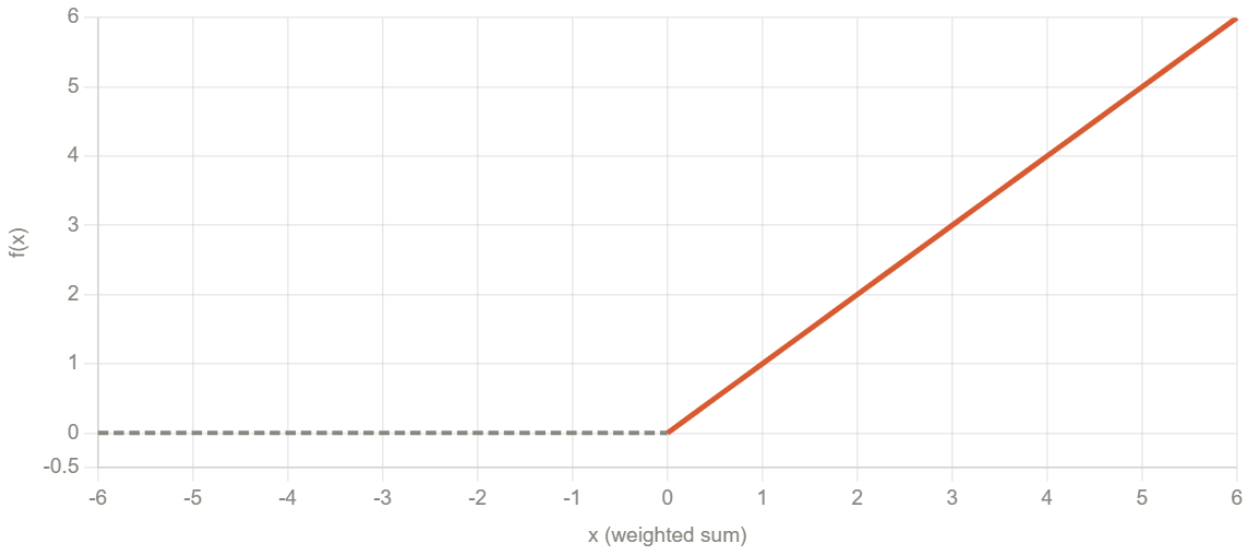
Advantages:

- The sigmoid function prevents the output from growing unboundedly, as the output of $f(x)$ remains between 0 and 1 even when the weighted sum grows.
- Saturated neurons like those in the sigmoid function are useful for binary classification tasks (like spam filters), as the output can be programmed to tend towards one of only two outputs.

Disadvantages

- Training is slower at its onset, as the network starts learning from the saturated regions, which do not produce meaningful patterns, because the curve is relatively flat in the saturated regions.
- The sigmoid function is computationally expensive, compared to the ReLU function. This is significant in larger operations where millions of neurons are being computed. The learning required for the saturated regions contributes to the expense and time taken.

ReLU function $f(x) = \max(0, x)$



a.) Input range that produces significant output changes

The range $x > 0$ creates the most significant output change, as the output grows linearly from there.

b.) Where the function saturates (if applicable)

The function only saturates at $x < 0$ (here the output is permanently 0). This illustrates the dying ReLU problem. ReLU does not saturate for positive inputs.

c.) Potential advantages/disadvantages in neural networks

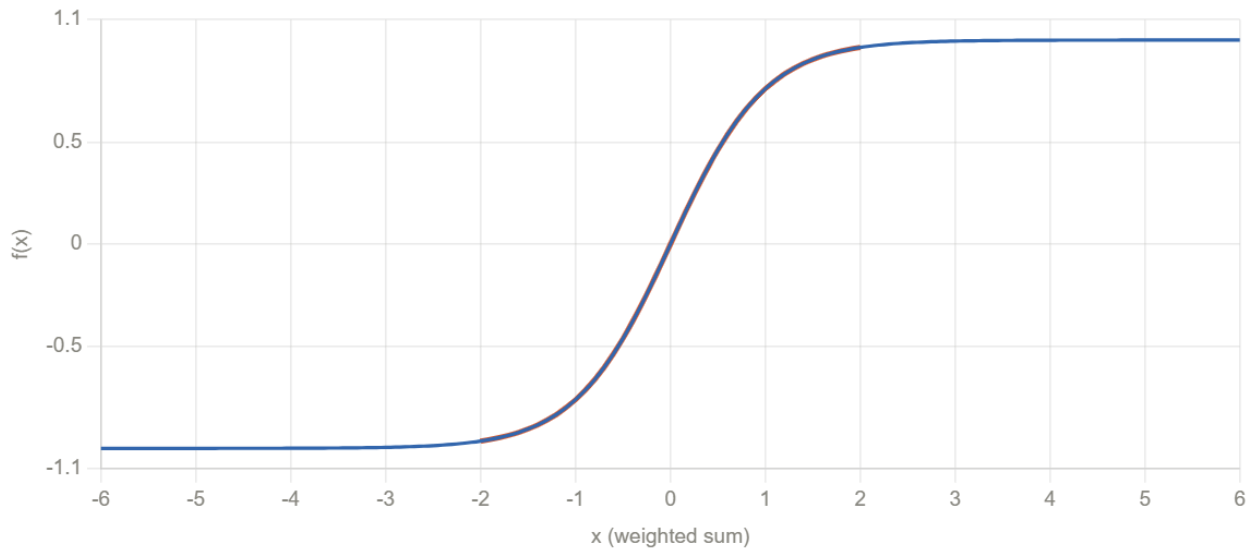
Advantages:

- There is no vanishing gradient for $x > 0$
- The function is computationally cheap, and training is much faster than with the Sigmoid function.

Disadvantages

- The Dying ReLU problem means that neurons with $x < 0$ have zero gradients and so stop learning permanently.
- The right side of the graph ($x > 0$) represents unbounded output, where $f(x)$ increases linearly indefinitely, which can cause exploding activations without adequate weight initialisation.

Tanh function $f(x) = (e^x - e^{-x}) / (e^x + e^{-x})$.



a.) Input range that produces significant output changes

The range $-2 < x < 2$ produces significant output changes in the range: $-1 < f(x) < 1$

b.) Where the function saturates (if applicable)

The function saturates at both ends ($x < -2$ and $x > 2$)

c.) Potential advantages/disadvantages in neural networks

Advantages:

- The function is zero-centered, as the primary outputs range from -1 to 1 . As such, the gradients can be positive or negative, which makes the function faster than sigmoid.
- The curve is symmetric around the origin, meaning negative inputs produce negative outputs, allowing the function to present a full range in many situations.

Disadvantages

- The function still saturates at both ends $x < -2$, and $x > 2$, which leads to vanishing gradients in deep networks.
- Like Sigmoid, the function is more computationally expensive than ReLU.

Activity 3: Network Architecture Design

Selected Problem for Neural Network: Classifying emails as spam or not spam

Input layer (what features and how many)

Features	Type	Notes
Links to known spam or phishing sites	Binary	
Deliberate obfuscation of known spam words	Binary	Text meant to evade filters like “fr£e”
Sender domain age	Continuous	Newer domains are more likely to be spam
Ratio of exclamation marks and capital letters	Continuous	Urgent-sounding text is more likely spam
Attachments present	Binary	Phishing emails could contain malware attachments
Time of day	Continuous	
Behavioural signals	Continuous	Whether other users reported the sender as spam

Hidden layers (how many and size)

- **Layer 1 (about 1000 to 5000 neurons):** captures frequency of common spam words (such as: click, free, now, hurry)
- **Layer 2 (512 neurons):** ReLU. Considers word patterns and structural features of the email. (Caps ratio, presence of URLs)
- **Layer 3 (128 neurons):** Funnel the factors into fewer high-level signals. Classifies the email as Phishing, Newsletter, Promotion, Regular email.
- **Output (1 neuron):** A single probability output: spam or not spam

Output layer (format and size)

The output is 1 neuron large (a binary feature 1 or 0 as in spam or not spam)

The output is in the form of a scalar continuous probability between 0 and 1. For example, 0.98 means certainly spam, and 0.23 means certainly not spam

Type of activation functions: ReLU and Sigmoid

- **Input and Processing: ReLU** is ideal for the input layer.
ReLU does not have vanishing gradients for positive inputs. It is also computationally cheap, which is ideal for an email filter system with thousands of inputs, working on hundreds of emails.
- **Output:** Sigmoid is ideal for the output as Sigmoid's $f(x)$ output is between 0 and 1, which works well for binary classification (spam or not spam)

Potential challenges in training this network

- **Concept drift:** Spam and phishing email practices change regularly: Senders continually attempt to evade modern spam filters, for example, by avoiding some words that are commonly associated with spam.
- **Cost Asymmetry between false positives and false negatives:** The negative effects of false positive (legitimate email being classed as spam) are often more than those of false negatives (spam being missed by the filters), for

example, because the email user could miss important emails. As such, the sigmoid threshold (which is between 0 and 1) is often not set at 0.5 in real spam filters, but slightly higher at 0.6 or 0.7.

- **Adversarial obfuscation:** Spam dispatchers adapt to spam filters by concealing the features of the email that the filter is designed to catch. For example, by including legitimate-sounding boilerplate text, or embedding text in images.

Deliverable: a sketch image of the architecture

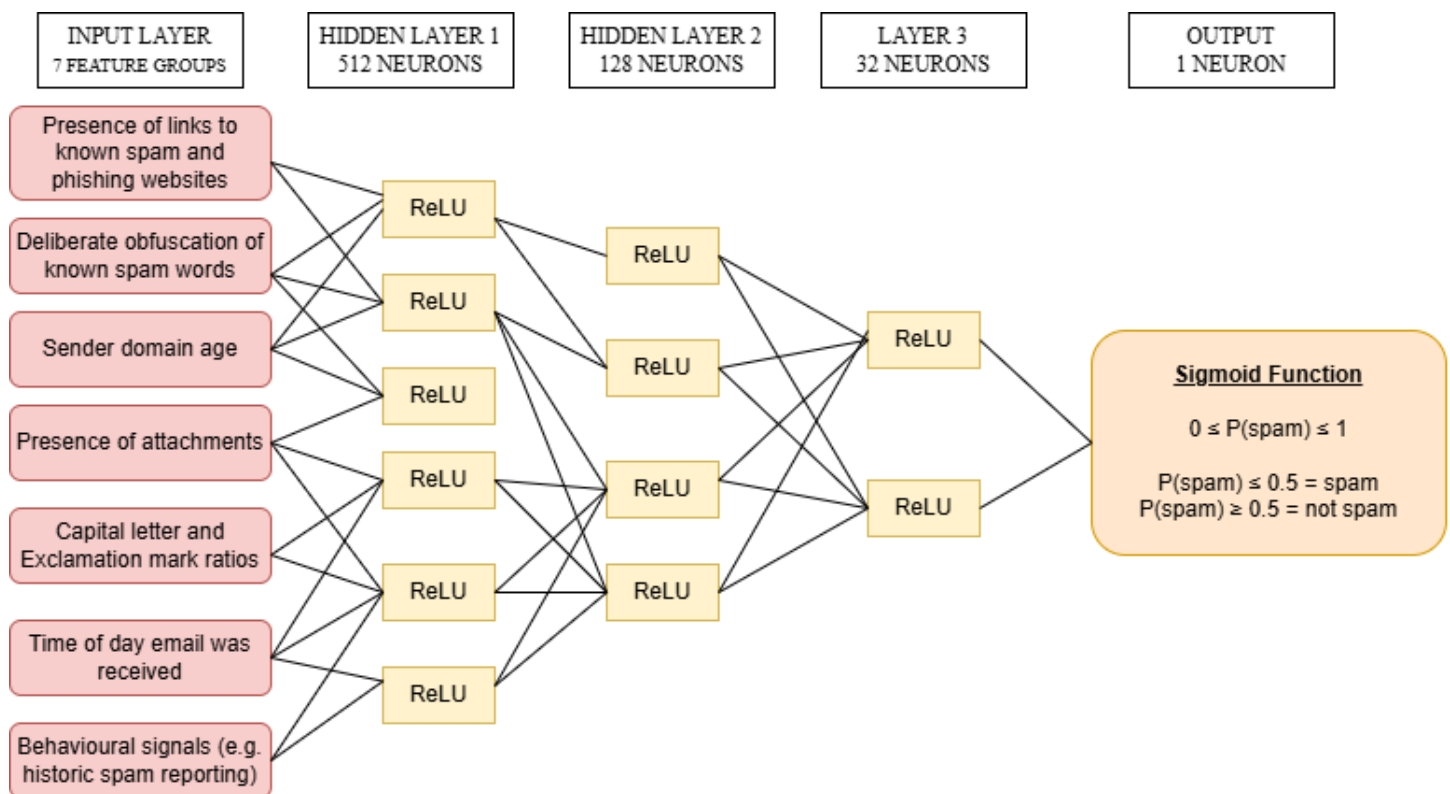


Diagram: **Sketch of the Spam filter network architecture.** Diagram prepared on: <https://app.diagrams.net/>

Bibliography

- Ahmed, N., Amin, R., Aldabbas, H., Koundal, D., Alouffi, B., & Shah, T. (2022). Machine learning techniques for spam detection in email and IoT platforms: Analysis and research challenges. *Security and Communication Networks*, 2022, Article 1862888. <https://doi.org/10.1155/2022/1862888>
- Dada, E. G., Bassi, J. S., Chiroma, H., Abdulhamid, S. M., Adetunmbi, A. O., & Ajibuwa, O. E. (2019). Machine learning for email spam filtering: Review, approaches and open research problems. *Heliyon*, 5(6), e01802. <https://doi.org/10.1016/j.heliyon.2019.e01802>